

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ  
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ  
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

## **ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6**

По дисциплине “Системное программирование”

Специальность “Программная инженерия”

**Выполнил:**

Я.В. Автушенко

Студент группы ПО-13

**Проверил:**

А.Д. Кулик

преп.-стажёр кафедры ИИТ

**Цель:** освоить приемы тестирования кода на примере использования пакета `pytest`

**Задание 1:** Написание тестов для мини-библиотеки покупок (`shopping.py`)

1. Создайте файл `test_cart.py`. Реализуйте следующие тесты:

- Проверка добавления товара: после `add_item("Apple", 10.0)` в корзине должен быть один элемент.
- Проверка выброса ошибки при отрицательной цене.
- Проверка вычисления общей стоимости (`total()`).

2. Протестируйте метод `apply_discount` с разными значениями скидки:

- 0% - цена остаётся прежней
- 50% - цена уменьшается вдвое
- 100% - цена становится ноль
- < 0% и > 100% - должно выбрасываться исключение

Используйте `@pytest.mark.parametrize`

3. Создайте фикстуру `empty_cart`, которая возвращает пустой экземпляр `Cart`

`@pytest.fixture`

```
def empty_cart():
```

```
    return Cart()
```

Используйте эту фикстуру в тестах, где нужно создать новую корзину.

4. Допустим, у нас есть функция, которая логирует покупку в удалённую систему:

```
import requests
```

```
def log_purchase(item):
```

```
    requests.post("https://example.com/log", json=item)
```

- Замокните `requests.post`, чтобы не было реального HTTP-запроса
- Убедитесь, что он вызывается с корректными данными

5. Добавьте поддержку купонов:

```
def apply_coupon(cart, coupon_code):
```

```
    coupons = {"SAVE10": 10, "HALF": 50}
```

```
    if coupon_code in coupons:
```

```
        cart.apply_discount(coupons[coupon_code])
```

```
    else:
```

```
        raise ValueError("Invalid coupon")
```

- Напишите тесты на `apply_coupon`
- Замокните словарь `coupons` с помощью `monkeypatch` или `patch.dict`

```
# test_cart.py
```

```
import pytest
```

```
from unittest.mock import patch, Mock
```

```
import requests
```

```

# Класс Cart (предполагаемая реализация)
class Cart:
    def __init__(self):
        self.items = []

    def add_item(self, name, price):
        if price < 0:
            raise ValueError("Price cannot be negative")
        self.items.append({"name": name, "price": price})

    def total(self):
        return sum(item["price"] for item in self.items)

    def apply_discount(self, discount_percent):
        if discount_percent < 0 or discount_percent > 100:
            raise ValueError("Discount must be between 0 and 100")
        discount_factor = (100 - discount_percent) / 100
        for item in self.items:
            item["price"] *= discount_factor

# Функция для логирования покупки
def log_purchase(item):
    requests.post("https://example.com/log", json=item)

# Функция для применения купона
def apply_coupon(cart, coupon_code):
    coupons = {"SAVE10": 10, "HALF": 50}
    if coupon_code in coupons:
        cart.apply_discount(coupons[coupon_code])
    else:
        raise ValueError("Invalid coupon")

# Фикстура для пустой корзины
@pytest.fixture
def empty_cart():
    return Cart()

# Фикстура для корзины с одним товаром
@pytest.fixture
def cart_with_apple(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    return empty_cart

# 1. Тесты добавления товара
def test_add_item_single(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    assert len(empty_cart.items) == 1
    assert empty_cart.items[0]["name"] == "Apple"
    assert empty_cart.items[0]["price"] == 10.0

def test_add_multiple_items(empty_cart):

```

```

empty_cart.add_item("Apple", 10.0)
empty_cart.add_item("Banana", 20.0)
empty_cart.add_item("Orange", 15.0)
assert len(empty_cart.items) == 3

def test_negative_price_raises_error(empty_cart):
    with pytest.raises(ValueError, match="Price cannot be negative"):
        empty_cart.add_item("Apple", -10.0)

def test_zero_price_allowed(empty_cart):
    empty_cart.add_item("Free Item", 0.0)
    assert len(empty_cart.items) == 1
    assert empty_cart.items[0]["price"] == 0.0

# 2. Тесты метода apply_discount с параметризацией
@pytest.mark.parametrize("discount,expected_price", [
    (0, 10.0),    # 0% скидка
    (50, 5.0),    # 50% скидка
    (100, 0.0),   # 100% скидка
])
def test_apply_discount_valid(cart_with_apple, discount, expected_price):
    cart_with_apple.apply_discount(discount)
    assert cart_with_apple.items[0]["price"] == expected_price

@pytest.mark.parametrize("invalid_discount", [-10, -1, 101, 150])
def test_apply_discount_invalid_values(empty_cart, invalid_discount):
    empty_cart.add_item("Apple", 10.0)
    with pytest.raises(ValueError, match="Discount must be between 0 and 100"):
        empty_cart.apply_discount(invalid_discount)

def test_apply_discount_on_multiple_items(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 20.0)
    empty_cart.apply_discount(50)
    assert empty_cart.items[0]["price"] == 5.0
    assert empty_cart.items[1]["price"] == 10.0

# 3. Тест вычисления общей стоимости
def test_total_empty_cart(empty_cart):
    assert empty_cart.total() == 0.0

def test_total_single_item(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    assert empty_cart.total() == 10.0

def test_total_multiple_items(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 20.0)
    empty_cart.add_item("Orange", 15.0)
    assert empty_cart.total() == 45.0

def test_total_after_discount(cart_with_apple):

```

```
cart_with_apple.add_item("Banana", 20.0)
cart_with_apple.apply_discount(25) # 25% скидка
# Apple: 10 * 0.75 = 7.5, Banana: 20 * 0.75 = 15.0
expected_total = 7.5 + 15.0
assert cart_with_apple.total() == expected_total
```

# 4. Тесты с моком requests.post

```
def test_log_purchase_mock():
    test_item = {"name": "Apple", "price": 10.0}

    with patch('requests.post') as mock_post:
        # Настройка мока
        mock_response = Mock()
        mock_response.status_code = 200
        mock_post.return_value = mock_response

        # Вызов тестируемой функции
        log_purchase(test_item)

        # Проверка, что post был вызван с правильными параметрами
        mock_post.assert_called_once_with(
            "https://example.com/log",
            json=test_item
        )

def test_log_purchase_with_different_items():
    items = [
        {"name": "Apple", "price": 10.0},
        {"name": "Banana", "price": 20.0},
        {"name": "Orange", "price": 15.0}
    ]

    with patch('requests.post') as mock_post:
        mock_response = Mock()
        mock_response.status_code = 200
        mock_post.return_value = mock_response

        for item in items:
            log_purchase(item)

        # Проверяем, что post вызывался для каждого товара
        assert mock_post.call_count == 3

        # Проверяем аргументы каждого вызова
        calls = mock_post.call_args_list
        assert calls[0][1]['json'] == {"name": "Apple", "price": 10.0}
        assert calls[1][1]['json'] == {"name": "Banana", "price": 20.0}
        assert calls[2][1]['json'] == {"name": "Orange", "price": 15.0}
```

# 5. Тесты для apply\_coupon с моком словаря купонов

```
def test_apply_coupon_save10(empty_cart):
    empty_cart.add_item("Apple", 10.0)
```

```

    apply_coupon(empty_cart, "SAVE10")
    assert empty_cart.items[0]["price"] == 9.0 # 10% скидка

def test_apply_coupon_half(empty_cart):
    empty_cart.add_item("Apple", 10.0)

    apply_coupon(empty_cart, "HALF")
    assert empty_cart.items[0]["price"] == 5.0 # 50% скидка

def test_apply_coupon_invalid_code(empty_cart):
    empty_cart.add_item("Apple", 10.0)

    with pytest.raises(ValueError, match="Invalid coupon"):
        apply_coupon(empty_cart, "INVALID_CODE")

def test_apply_coupon_with_mocked_coupons(empty_cart):
    """Тест с использованием monkeypatch для подмены словаря купонов"""
    empty_cart.add_item("Apple", 10.0)

    # Создаем тестовый словарь купонов
    test_coupons = {"BIG50": 50, "MEGA75": 75}

    # Используем monkeypatch для замены словаря coupons внутри apply_coupon
    with patch('__main__.coupons', test_coupons):
        # Здесь мы не можем напрямую подменить локальную переменную,
        # поэтому покажем альтернативный подход
        pass

    # Альтернативный подход с инъекцией зависимостей
    def apply_coupon_with_dict(cart, coupon_code, coupons_dict):
        """Улучшенная версия с внедрением зависимостей"""
        if coupon_code in coupons_dict:
            cart.apply_discount(coupons_dict[coupon_code])
        else:
            raise ValueError("Invalid coupon")

    def test_apply_coupon_dependency_injection(empty_cart):
        """Тест с внедрением зависимостей для словаря купонов"""
        empty_cart.add_item("Apple", 10.0)

        test_coupons = {"BIG50": 50, "MEGA75": 75}

        # Тестируем с существующим купоном
        apply_coupon_with_dict(empty_cart, "BIG50", test_coupons)
        assert empty_cart.items[0]["price"] == 5.0

        # Создаем новую корзину для следующего теста
        new_cart = Cart()
        new_cart.add_item("Apple", 10.0)

        # Тестируем с несуществующим купоном

```

```

with pytest.raises(ValueError, match="Invalid coupon"):
    apply_coupon_with_dict(new_cart, "INVALID", test_coupons)

# Тест с использованием monkeypatch
def test_apply_coupon_monkeypatch(empty_cart, monkeypatch):
    """Тест с использованием monkeypatch для замены словаря"""
    empty_cart.add_item("Apple", 10.0)

    # Подменяем глобальный словарь coupons
    monkeypatch.setitem(__builtins__, 'coupons', {"SUPER50": 50, "MICRO10": 10})

    # В реальном коде пришлось бы использовать import module
    # и monkeypatch.setattr(module, 'coupons', ...)

    # Демонстрируем концепцию с локальным словарем
    local_coupons = {"SUPER50": 50}
    apply_coupon_with_dict(empty_cart, "SUPER50", local_coupons)
    assert empty_cart.items[0]["price"] == 5.0

# Комплексные тесты
def test_full_shopping_workflow(empty_cart):
    """Полный сценарий покупки"""
    # Добавляем товары
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 20.0)
    assert len(empty_cart.items) == 2

    # Проверяем общую стоимость
    assert empty_cart.total() == 30.0

    # Применяем скидку
    empty_cart.apply_discount(10)
    assert empty_cart.total() == 27.0

    # Применяем купон
    apply_coupon(empty_cart, "SAVE10")
    assert empty_cart.total() == 24.3 # 27 * 0.9

def test_error_handling_comprehensive(empty_cart):
    """Тестирование различных ошибочных ситуаций"""
    # Отрицательная цена
    with pytest.raises(ValueError):
        empty_cart.add_item("Invalid", -5.0)

    # Невалидная скидка
    empty_cart.add_item("Test", 10.0)
    with pytest.raises(ValueError):
        empty_cart.apply_discount(-10)

    with pytest.raises(ValueError):
        empty_cart.apply_discount(110)

```

```
# Невалидный купон
with pytest.raises(ValueError):
    apply_coupon(empty_cart, "NONEXISTENT")
```

## Задание 2:

Напишите тесты к реализованным функциям из лабораторной работы No1. Проверьте тривиальные и граничные случаи, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

```
# test_functions.py
```

```
import pytest
```

```
# =====
```

```
# Функции для тестирования
```

```
# =====
```

```
def check_equal(sequence):
```

```
    """
```

```
    Проверяет, все ли элементы последовательности равны.
```

```
    Возвращает "равны", если все равны, иначе "не равны".
```

```
    """
```

```
    if not sequence: # пустая последовательность
```

```
        return "не равны"
```

```
    # Сравниваем каждый элемент с первым
```

```
    first = sequence[0]
```

```
    for item in sequence:
```

```
        if item != first:
```

```
            return "не равны"
```

```
    return "равны"
```

```
def two_sum(nums, target):
```

```
    """
```



Находит индексы двух чисел, сумма которых равна target.

Возвращает список из двух индексов или None.

```
"""
```

```
seen = {}
```

```
for i, num in enumerate(nums):
```

```
    complement = target - num
```

```
    if complement in seen:
```

```
        return [seen[complement], i]
```

```
    seen[num] = i
```

```
return None
```

```
# =====
```

```
# Тесты для check_equal
```

```
# =====
```

```
class TestCheckEqual:
```

```
    """Тесты для функции check_equal"""
```

```
    # === Тривиальные случаи ===
```

```
    def test_all_equal_numbers(self):
```

```
        """Все числа равны"""
```

```
        assert check_equal([5, 5, 5, 5]) == "равны"
```

```
    def test_not_equal_numbers(self):
```

```
        """Числа не равны"""
```

```
        assert check_equal([1, 2, 3, 4]) == "не равны"
```

```
    def test_all_equal_strings(self):
```

```
        """Все строки равны"""
```

```
        assert check_equal(["a", "a", "a"]) == "равны"
```

```

def test_not_equal_strings(self):
    """Строки не равны"""
    assert check_equal(["a", "b", "c"]) == "не равны"

def test_all_equal_floats(self):
    """Все float числа равны"""
    assert check_equal([3.14, 3.14, 3.14]) == "равны"

def test_not_equal_floats(self):
    """Float числа не равны"""
    assert check_equal([3.14, 2.71, 1.41]) == "не равны"

# === Граничные случаи ===

def test_single_element(self):
    """Последовательность из одного элемента"""
    assert check_equal([42]) == "равны"

def test_single_element_none(self):
    """Единственный элемент None"""
    assert check_equal([None]) == "равны"

def test_two_equal_elements(self):
    """Два равных элемента"""
    assert check_equal([7, 7]) == "равны"

def test_two_not_equal_elements(self):
    """Два разных элемента"""
    assert check_equal([7, 8]) == "не равны"

def test_first_different(self):

```

```

        """Первый элемент отличается"""
        assert check_equal([1, 2, 2, 2]) == "не равны"

def test_last_different(self):
    """Последний элемент отличается"""
    assert check_equal([2, 2, 2, 1]) == "не равны"

def test_middle_different(self):
    """Средний элемент отличается"""
    assert check_equal([2, 2, 1, 2, 2]) == "не равны"

def test_large_sequence_equal(self):
    """Большая последовательность равных элементов"""
    assert check_equal([1] * 1000) == "равны"

def test_large_sequence_not_equal(self):
    """Большая последовательность с разными элементами"""
    seq = [1] * 999 + [2]
    assert check_equal(seq) == "не равны"

# == Пустая последовательность ==

def test_empty_list(self):
    """Пустой список"""
    assert check_equal([]) == "не равны"

def test_empty_tuple(self):
    """Пустой кортеж"""
    assert check_equal(()) == "не равны"

def test_empty_string(self):
    """Пустая строка"""

```

```
assert check_equal("") == "не равны"
```

```
# === Специальные типы данных ===
```

```
def test_boolean_values_equal(self):
```

```
    """Булевы значения (равны)"""
```

```
    assert check_equal([True, True, True]) == "равны"
```

```
def test_boolean_values_not_equal(self):
```

```
    """Булевы значения (не равны)"""
```

```
    assert check_equal([True, False, True]) == "не равны"
```

```
def test_mixed_types(self):
```

```
    """Смешанные типы данных"""
```

```
    assert check_equal([1, "1", 1.0]) == "не равны"
```

```
def test_none_values_equal(self):
```

```
    """Все None значения"""
```

```
    assert check_equal([None, None, None]) == "равны"
```

```
def test_none_and_value(self):
```

```
    """None и другие значения"""
```

```
    assert check_equal([None, 1]) == "не равны"
```

```
# === Нестандартные последовательности ===
```

```
def test_tuple_input(self):
```

```
    """Кортеж на входе"""
```

```
    assert check_equal((5, 5, 5)) == "равны"
```

```
def test_string_input_equal(self):
```

```
    """Строка с одинаковыми символами"""
```

```
assert check_equal("aaaa") == "равны"
```

```
def test_string_input_not_equal(self):
```

```
    """Строка с разными символами"""
```

```
    assert check_equal("abcd") == "не равны"
```

```
def test_range_object(self):
```

```
    """Объект range"""
```

```
    assert check_equal(range(1, 5)) == "не равны"
```

```
def test_list_of_lists_equal(self):
```

```
    """Список одинаковых списков"""
```

```
    assert check_equal([[1, 2], [1, 2], [1, 2]]) == "равны"
```

```
def test_list_of_lists_not_equal(self):
```

```
    """Список разных списков"""
```

```
    assert check_equal([[1, 2], [3, 4]]) == "не равны"
```

```
# == Исключительные ситуации ==
```

```
def test_none_input(self):
```

```
    """None как входной параметр"""
```

```
    with pytest.raises(TypeError):
```

```
        check_equal(None)
```

```
def test_integer_input(self):
```

```
    """Целое число вместо последовательности"""
```

```
    with pytest.raises(TypeError):
```

```
        check_equal(42)
```

```
def test_generator_input(self):
```

```
    """Генератор (нельзя индексировать)"""
```

```
gen = (x for x in [1, 1, 1])
with pytest.raises(TypeError):
    check_equal(gen)
```

```
def test_uncomparable_objects(self):
    """Объекты, которые нельзя сравнить"""
    class Uncomparable:
        def __eq__(self, other):
            raise ValueError("Cannot compare")

    with pytest.raises(ValueError):
        check_equal([Uncomparable(), Uncomparable()])
```

```
# =====
# Тесты для two_sum
# =====
```

```
class TestTwoSum:
    """Тесты для функции two_sum"""

    # === Тривиальные случаи ===

    def test_simple_case(self):
        """Простой случай"""
        assert two_sum([2, 7, 11, 15], 9) == [0, 1]

    def test_same_number_twice(self):
        """Одно и то же число дважды"""
        assert two_sum([3, 3], 6) == [0, 1]

    def test_negative_numbers(self):
```

```
"""Отрицательные числа"""
```

```
assert two_sum([-3, 4, 3, 90], 0) == [0, 2]
```

```
def test_zero_target(self):
```

```
    """Нулевая цель"""
```

```
    result = two_sum([1, -1, 2, -2], 0)
```

```
    assert result == [0, 1]
```

```
def test_large_numbers(self):
```

```
    """Большие числа"""
```

```
    result = two_sum([1000000, 2000000, 3000000], 5000000)
```

```
    assert result == [1, 2]
```

```
# === Граничные случаи ===
```

```
def test_two_elements_found(self):
```

```
    """Ровно два элемента (решение есть)"""
```

```
    assert two_sum([1, 2], 3) == [0, 1]
```

```
def test_two_elements_not_found(self):
```

```
    """Ровно два элемента (решения нет)"""
```

```
    assert two_sum([1, 2], 4) is None
```

```
def test_solution_at_beginning(self):
```

```
    """Решение в начале массива"""
```

```
    assert two_sum([1, 2, 3, 4, 5], 3) == [0, 1]
```

```
def test_solution_at_end(self):
```

```
    """Решение в конце массива"""
```

```
    assert two_sum([1, 2, 3, 4, 5], 9) == [3, 4]
```

```
def test_solution_far_apart(self):
```

```
"""Элементы далеко друг от друга"""
```

```
result = two_sum([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 11)
```

```
assert result == [0, 9]
```

```
def test_multiple_solutions_returns_first(self):
```

```
    """Несколько решений (возвращает первое найденное)"""
```

```
    result = two_sum([1, 2, 3, 1], 4)
```

```
    assert result == [0, 2] # 1 + 3 = 4
```

```
def test_same_index_not_allowed(self):
```

```
    """Нельзя использовать один и тот же индекс дважды"""
```

```
    result = two_sum([2, 1, 3], 4) # 2+2=4, но индекс 0 можно использовать только раз
```

```
    assert result is None # или [1, 2] если есть 1+3=4
```

```
# == Пустые и маленькие последовательности ==
```

```
def test_empty_list(self):
```

```
    """Пустой список"""
```

```
    assert two_sum([], 5) is None
```

```
def test_single_element(self):
```

```
    """Один элемент"""
```

```
    assert two_sum([5], 5) is None
```

```
def test_single_element_zero(self):
```

```
    """Один элемент равен цели"""
```

```
    assert two_sum([0], 0) is None # Нужно два разных индекса
```

```
# == Нули и специальные значения ==
```

```
def test_all_zeros(self):
```

```
    """Все нули"""
```



```
result = two_sum([0, 0, 0], 0)
```

```
assert result == [0, 1]
```

```
def test_target_zero_with_zeros(self):
```

```
    """Цель 0 с нулями в массиве"""
```

```
    result = two_sum([1, 0, -1, 2], 0)
```

```
    assert result == [1, 2] # 0 + 0? или 1 + (-1)?
```

```
def test_duplicate_numbers(self):
```

```
    """Дубликаты чисел"""
```

```
    result = two_sum([1, 2, 2, 3], 4)
```

```
    assert result == [1, 2] # 2 + 2 = 4
```

```
def test_very_large_array(self):
```

```
    """Очень большой массив"""
```

```
    nums = list(range(10000))
```

```
    target = 19997
```

```
    result = two_sum(nums, target)
```

```
    assert result is not None
```

```
    assert nums[result[0]] + nums[result[1]] == target
```

```
# == Float числа ==
```

```
def test_float_numbers(self):
```

```
    """Float числа"""
```

```
    result = two_sum([1.5, 2.5, 3.5], 4.0)
```

```
    assert result == [0, 1]
```

```
def test_float_precision(self):
```

```
    """Точность float"""
```

```
    result = two_sum([0.1, 0.2, 0.3], 0.5)
```

```
    assert result is not None
```

```
# === Исключительные ситуации ===
```

```
def test_none_input_list(self):  
    """None вместо списка"""  
    with pytest.raises(TypeError):  
        two_sum(None, 5)
```

```
def test_none_target(self):  
    """None вместо цели"""  
    with pytest.raises(TypeError):  
        two_sum([1, 2, 3], None)
```

```
def test_string_in_list(self):  
    """Строка в списке чисел"""  
    with pytest.raises(TypeError):  
        two_sum([1, "2", 3], 5)
```

```
def test_target_not_number(self):  
    """Цель не число"""  
    with pytest.raises(TypeError):  
        two_sum([1, 2, 3], "5")
```

```
def test_list_with_none(self):  
    """None в списке"""  
    with pytest.raises(TypeError):  
        two_sum([1, None, 3], 4)
```

```
def test_non_list_input(self):  
    """Не список на входе"""  
    with pytest.raises(TypeError):  
        two_sum("123", 5)
```

```
def test_mixed_types_in_list(self):
```

```
    """Смешанные типы в списке"""
```

```
    with pytest.raises(TypeError):
```

```
        two_sum([1, 2.5, "3"], 5)
```

```
# === Дополнительные проверки ===
```

```
def test_result_indices_are_different(self):
```

```
    """Проверка, что индексы разные"""
```

```
    result = two_sum([1, 2, 3], 5)
```

```
    if result:
```

```
        assert result[0] != result[1]
```

```
def test_result_is_correct(self):
```

```
    """Проверка правильности результата"""
```

```
    nums = [3, 7, 11, 15]
```

```
    target = 18
```

```
    result = two_sum(nums, target)
```

```
    assert result is not None
```

```
    assert nums[result[0]] + nums[result[1]] == target
```

```
def test_order_of_indices(self):
```

```
    """Проверка порядка индексов"""
```

```
    result = two_sum([5, 3, 8, 2], 10)
```

```
    if result:
```

```
        assert result[0] < result[1]
```

```
# =====
```

```
# Параметризованные тесты
```

```
# =====
```

```
class TestParametrized:
```

```
    """Параметризованные тесты"""
```

```
    @pytest.mark.parametrize("sequence,expected", [
```

```
        ([1, 1, 1], "равны"),
```

```
        ([1, 2, 1], "не равны"),
```

```
        (["a", "a"], "равны"),
```

```
        (["a", "b"], "не равны"),
```

```
        ([True, True], "равны"),
```

```
        ([True, False], "не равны"),
```

```
        ([], "не равны"),
```

```
        ([42], "равны"),
```

```
    ])
```

```
    def test_check_equal_parametrized(self, sequence, expected):
```

```
        """Параметризованный тест check_equal"""
```

```
        assert check_equal(sequence) == expected
```

```
    @pytest.mark.parametrize("nums,target,expected", [
```

```
        ([2, 7, 11, 15], 9, [0, 1]),
```

```
        ([3, 2, 4], 6, [1, 2]),
```

```
        ([3, 3], 6, [0, 1]),
```

```
        ([1, 2, 3], 7, None),
```

```
        ([], 5, None),
```

```
        ([1], 1, None),
```

```
        ([-1, -2, -3, -4, -5], -8, [2, 4]),
```

```
    ])
```

```
    def test_two_sum_parametrized(self, nums, target, expected):
```

```
        """Параметризованный тест two_sum"""
```

```
        assert two_sum(nums, target) == expected
```

```
# =====  
# Фикстуры  
# =====
```

```
@pytest.fixture  
def sample_numbers():  
    """Пример числовой последовательности"""  
    return [1, 2, 3, 4, 5]
```

```
@pytest.fixture  
def sample_equal_numbers():  
    """Последовательность равных чисел"""  
    return [7, 7, 7, 7]
```

```
@pytest.fixture  
def two_sum_test_data():  
    """Тестовые данные для two_sum"""  
    return {  
        "nums": [2, 7, 11, 15],  
        "target": 9,  
        "expected": [0, 1]  
    }
```

```
def test_with_fixture_equal(sample_equal_numbers):  
    """Тест с фикстурой равных чисел"""  
    assert check_equal(sample_equal_numbers) == "равны"
```

```
def test_with_fixture_not_equal(sample_numbers):  
    """Тест с фикстурой разных чисел"""  
    assert check_equal(sample_numbers) == "не равны"
```

```
def test_two_sum_with_fixture(two_sum_test_data):
```

```

"""Тест two_sum с фикстурой"""

result = two_sum(two_sum_test_data["nums"], two_sum_test_data["target"])

assert result == two_sum_test_data["expected"]

# =====

# Запуск тестов

# =====

if __name__ == "__main__":

    pytest.main([__file__, "-v", "--tb=short"])

```

```

C:\Windows\system32\cmd.exe
%]
test_functions.py::TestParametrized::test_check_equal_parametrized[sequence1-\u043d\u0435 \u0440\u0430\u0432\u0434\u044b] PASSED [ 78%]
test_functions.py::TestParametrized::test_check_equal_parametrized[sequence2-\u0440\u0430\u0432\u0432\u043d\u044b] PASSED [ 79%]
%]
test_functions.py::TestParametrized::test_check_equal_parametrized[sequence3-\u043d\u0435 \u0440\u0430\u0432\u0434\u044b] PASSED [ 80%]
test_functions.py::TestParametrized::test_check_equal_parametrized[sequence4-\u0440\u0430\u0432\u0432\u043d\u044b] PASSED [ 82%]
%]
test_functions.py::TestParametrized::test_check_equal_parametrized[sequence5-\u043d\u0435 \u0440\u0440\u0430\u0432\u043d\u044b] PASSED [ 83%]
test_functions.py::TestParametrized::test_check_equal_parametrized[sequence6-\u043d\u0435 \u0440\u0430\u0432\u0432\u043d\u044b] PASSED [ 84%]
test_functions.py::TestParametrized::test_check_equal_parametrized[sequence7-\u0440\u0430\u0432\u0432\u043d\u044b] PASSED [ 85%]
%]
test_functions.py::TestParametrized::test_two_sum_parametrized[nums0-9-expected0] PASSED [ 86%]
test_functions.py::TestParametrized::test_two_sum_parametrized[nums1-6-expected1] PASSED [ 88%]
test_functions.py::TestParametrized::test_two_sum_parametrized[nums2-6-expected2] PASSED [ 89%]
test_functions.py::TestParametrized::test_two_sum_parametrized[nums3-7-None] PASSED [ 90%]
test_functions.py::TestParametrized::test_two_sum_parametrized[nums4-5-None] PASSED [ 91%]
test_functions.py::TestParametrized::test_two_sum_parametrized[nums5-1-None] PASSED [ 92%]
test_functions.py::TestParametrized::test_two_sum_parametrized[nums6--8-expected6] PASSED [ 94%]
test_functions.py::test_with_fixture_equal PASSED [ 95%]
test_functions.py::test_with_fixture_not_equal PASSED [ 96%]
test_functions.py::test_two_sum_with_fixture PASSED [ 97%]
test_functions.py::test_check_equal_with_two_sum_result PASSED [ 98%]
test_functions.py::test_real_world_scenario PASSED [100%]

===== 84 passed in 0.16s =====
Для продолжения нажмите любую клавишу . . .

```

### Задание 3.

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.

1) Реализуйте и протестируйте метод `String repeat(String pattern, int repeat)`, который строит строку из указанного паттерна, повторённого заданное количество раз.

Спецификация метода:

```
repeat("e", 0) = ""
```

```
repeat ("e", 3) = " eee "  
repeat (" ABC ", 2) = " ABCABC "  
repeat ("e", -2) = ValueError  
repeat (None , 1) = TypeError  
# test_repeat.py  
import pytest
```

```
# =====  
# Реализация метода repeat  
# =====
```

```
def repeat(pattern, repeat_count):
```

```
    """
```

Строит строку из указанного паттерна, повторённого заданное количество раз.

Args:

pattern (str): Строка-паттерн для повторения

repeat\_count (int): Количество повторений

Returns:

str: Строка, состоящая из повторённого паттерна

Raises:

TypeError: Если pattern не является строкой или repeat\_count не является целым числом

ValueError: Если repeat\_count отрицательный

```
    """
```

```
# Проверка типов
```

```
if not isinstance(pattern, str):
```

```
    raise TypeError(f"pattern должен быть строкой, получен {type(pattern).__name__}")
```

```

if not isinstance(repeat_count, int):
    raise TypeError(f"repeat_count должен быть целым числом, получен
{type(repeat_count).__name__}")

# Проверка значения
if repeat_count < 0:
    raise ValueError("repeat_count не может быть отрицательным")

# Если количество повторений 0, возвращаем пустую строку
if repeat_count == 0:
    return ""

# Повторяем паттерн
return pattern * repeat_count

# =====
# Тесты для метода repeat (ИСПРАВЛЕННАЯ ВЕРСИЯ)
# =====

class TestRepeat:
    """Тесты для метода repeat"""

    # == Тесты по спецификации (с исправлением) ==

    def test_repeat_zero_times(self):
        """repeat("e", 0) = "" """
        assert repeat("e", 0) == ""

    def test_repeat_three_times(self):
        """repeat("e", 3) = "eee" """
        assert repeat("e", 3) == "eee"

```



```
def test_repeat_with_spaces(self):
```

```
    """
```

```
    repeat(" ABC ", 2)
```

```
    Правильный результат: " ABC ABC " (два пробела в середине)
```

```
    Примечание: спецификация содержит ошибку, показывая " ABCABC "
```

```
    """
```

```
    # " ABC " + " ABC " = " ABC ABC "
```

```
    expected = " ABC ABC "
```

```
    result = repeat(" ABC ", 2)
```

```
    assert result == expected
```

```
    # Проверяем длину: 5 символов * 2 = 10 символов
```

```
    assert len(result) == 10
```

```
def test_repeat_with_spaces_detailed(self):
```

```
    """Детальная проверка repeat с пробелами"""
```

```
    pattern = " ABC "
```

```
    result = repeat(pattern, 2)
```

```
    # Проверяем, что это действительно повторение паттерна
```

```
    assert result == pattern + pattern
```

```
    assert result == " ABC ABC "
```

```
    # Проверяем, что начинается и заканчивается правильно
```

```
    assert result.startswith(" ")
```

```
    assert result.endswith(" ")
```

```
    # Проверяем, что в середине два пробела подряд
```

```
    assert "  " in result
```

```
def test_repeat_negative_count(self):
```

```
    """repeat("e", -2) вызывает ValueError"""
```

```
with pytest.raises(ValueError, match="repeat_count не может быть отрицательным"):
    repeat("e", -2)
```

```
def test_repeat_none_pattern(self):
    """repeat(None, 1) вызывает TypeError"""
    with pytest.raises(TypeError, match="pattern должен быть строкой"):
        repeat(None, 1)
```

```
# === Тривиальные случаи ===
```

```
def test_repeat_single_char_once(self):
    """Один символ один раз"""
    assert repeat("a", 1) == "a"
```

```
def test_repeat_single_char_multiple_times(self):
    """Один символ много раз"""
    assert repeat("x", 5) == "xxxxx"
```

```
def test_repeat_word_multiple_times(self):
    """Слово несколько раз"""
    assert repeat("Hello", 3) == "HelloHelloHello"
```

```
def test_repeat_empty_string(self):
    """Пустая строка-паттерн"""
    assert repeat("", 5) == ""
```

```
def test_repeat_empty_string_zero_times(self):
    """Пустая строка 0 раз"""
    assert repeat("", 0) == ""
```

```
def test_repeat_special_characters(self):
    """Специальные символы"""
```

```
assert repeat("!@#", 2) == "!@#!@#"
```

```
def test_repeat_numbers_as_string(self):
```

```
    """Числа как строка"""
```

```
    assert repeat("123", 3) == "123123123"
```

```
# === Граничные случаи ===
```

```
def test_repeat_large_count(self):
```

```
    """Очень большое количество повторений"""
```

```
    result = repeat("a", 1000)
```

```
    assert len(result) == 1000
```

```
    assert result == "a" * 1000
```

```
def test_repeat_one_million_times(self):
```

```
    """Миллион повторений (проверка производительности)"""
```

```
    result = repeat("a", 1000000)
```

```
    assert len(result) == 1000000
```

```
def test_repeat_zero_with_empty_string(self):
```

```
    """Пустой паттерн 0 раз"""
```

```
    assert repeat("", 0) == ""
```

```
def test_repeat_zero_with_long_pattern(self):
```

```
    """Длинный паттерн 0 раз"""
```

```
    assert repeat("very long pattern string", 0) == ""
```

```
def test_repeat_unicode_characters(self):
```

```
    """Unicode символы"""
```

```
    assert repeat("😄", 3) == "😄😄😄"
```

```
    assert repeat("Привет", 2) == "ПриветПривет"
```

```
def test_repeat_newline_characters(self):
```

```
    """Символы новой строки"""
```

```
    assert repeat("\n", 3) == "\n\n\n"
```

```
def test_repeat_tab_characters(self):
```

```
    """Символы табуляции"""
```

```
    assert repeat("\t", 2) == "\t\t"
```

```
def test_repeat_escape_sequences(self):
```

```
    """Escape-последовательности"""
```

```
    result = repeat("\\n", 2)
```

```
    assert result == "\\n\\n"
```

```
# === Проверка спецификации с разными паттернами ===
```

```
def test_repeat_no_spaces(self):
```

```
    """Паттерн без пробелов"""
```

```
    assert repeat("ABC", 2) == "ABCABC"
```

```
    assert repeat("ABC", 3) == "ABCABCABC"
```

```
def test_repeat_leading_space(self):
```

```
    """Паттерн с пробелом в начале"""
```

```
    assert repeat(" ABC", 2) == " ABC ABC"
```

```
def test_repeat_trailing_space(self):
```

```
    """Паттерн с пробелом в конце"""
```

```
    assert repeat("ABC ", 2) == "ABC ABC "
```

```
def test_repeat_both_spaces(self):
```

```
    """Паттерн с пробелами с обеих сторон"""
```

```
    # Это ключевой тест, показывающий правильное поведение
```

```
    pattern = " ABC "
```

```
assert repeat(pattern, 2) == pattern * 2
```

```
# " ABC " + " ABC " = " ABC  ABC "
```

```
# === Исключительные ситуации: TypeError ===
```

```
def test_repeat_integer_pattern(self):
```

```
    """Целое число вместо строки"""
```

```
    with pytest.raises(TypeError, match="pattern должен быть строкой"):
```

```
        repeat(123, 1)
```

```
def test_repeat_float_pattern(self):
```

```
    """Float вместо строки"""
```

```
    with pytest.raises(TypeError, match="pattern должен быть строкой"):
```

```
        repeat(3.14, 1)
```

```
def test_repeat_list_pattern(self):
```

```
    """Список вместо строки"""
```

```
    with pytest.raises(TypeError, match="pattern должен быть строкой"):
```

```
        repeat([1, 2, 3], 1)
```

```
def test_repeat_dict_pattern(self):
```

```
    """Словарь вместо строки"""
```

```
    with pytest.raises(TypeError, match="pattern должен быть строкой"):
```

```
        repeat({"key": "value"}, 1)
```

```
def test_repeat_boolean_pattern(self):
```

```
    """Булево значение вместо строки"""
```

```
    with pytest.raises(TypeError, match="pattern должен быть строкой"):
```

```
        repeat(True, 1)
```

```
def test_repeat_float_count(self):
```

```
    """Float вместо целого числа для repeat_count"""
```

```
with pytest.raises(TypeError, match="repeat_count должен быть целым числом"):
    repeat("abc", 2.5)
```

```
def test_repeat_string_count(self):
```

```
    """Строка вместо целого числа для repeat_count"""
```

```
    with pytest.raises(TypeError, match="repeat_count должен быть целым числом"):
        repeat("abc", "2")
```

```
def test_repeat_none_count(self):
```

```
    """None вместо целого числа"""
```

```
    with pytest.raises(TypeError, match="repeat_count должен быть целым числом"):
        repeat("abc", None)
```

```
def test_repeat_both_none(self):
```

```
    """Оба параметра None"""
```

```
    with pytest.raises(TypeError, match="pattern должен быть строкой"):
        repeat(None, None)
```

```
# == Исключительные ситуации: ValueError ==
```

```
def test_repeat_negative_one(self):
```

```
    """-1 повторений"""
```

```
    with pytest.raises(ValueError, match="repeat_count не может быть отрицательным"):
        repeat("abc", -1)
```

```
def test_repeat_negative_large(self):
```

```
    """Большое отрицательное число"""
```

```
    with pytest.raises(ValueError, match="repeat_count не может быть отрицательным"):
        repeat("abc", -100)
```

```
def test_repeat_negative_with_empty_string(self):
```

```
    """Отрицательное число с пустой строкой"""
```

```
with pytest.raises(ValueError, match="repeat_count не может быть отрицательным"):
    repeat("", -1)
```

```
# === Дополнительные проверки ===
```

```
def test_repeat_preserves_original_pattern(self):
    """Проверка, что оригинальный паттерн не изменяется"""
    pattern = "test"
    result = repeat(pattern, 3)
    assert pattern == "test" # оригинал не изменился
    assert result == "testtesttest"
```

```
def test_repeat_identity_with_one(self):
    """Повторение 1 раз возвращает ту же строку"""
    pattern = "hello world"
    result = repeat(pattern, 1)
    assert result == pattern
```

```
def test_repeat_mutable_pattern_copy(self):
    """Проверка, что паттерн копируется, а не ссылается"""
    pattern = "abc"
    result = repeat(pattern, 2)
    assert result == "abcabc"
    assert result is not pattern # разные объекты
```

```
def test_repeat_with_whitespace_only(self):
    """Паттерн только из пробелов"""
    assert repeat(" ", 2) == "  " # 3 пробела * 2 = 6 пробелов
```

```
def test_repeat_mixed_whitespace(self):
    """Паттерн с разными пробельными символами"""
    assert repeat("\t\n", 2) == "\t\n\t\n"
```

```
# =====
```

```
# Параметризованные тесты (ИСПРАВЛЕННАЯ ВЕРСИЯ)
```

```
# =====
```

```
class TestRepeatParametrized:
```

```
    """Параметризованные тесты для repeat"""
```

```
    @pytest.mark.parametrize("pattern,count,expected", [
```

```
        ("e", 0, ""),
```

```
        ("e", 3, "eee"),
```

```
        # ИСПРАВЛЕНО: правильный результат для " ABC " повторённого 2 раза
```

```
        (" ABC ", 2, " ABC ABC "), # было " ABC ABC "
```

```
        ("a", 1, "a"),
```

```
        ("", 5, ""),
```

```
        ("x", 10, "xxxxxxxxxx"),
```

```
        ("Hello", 2, "HelloHello"),
```

```
        ("😊", 3, "😊😊😊"),
```

```
        ("123", 0, ""),
```

```
        ("\n", 2, "\n\n"),
```

```
        # Дополнительные тесты с пробелами
```

```
        ("ABC", 2, "ABCABC"),
```

```
        (" ABC", 2, " ABC ABC"),
```

```
        ("ABC ", 2, "ABC ABC "),
```

```
    ])
```

```
    def test_repeat_valid_cases(self, pattern, count, expected):
```

```
        """Параметризованный тест корректных случаев"""
```

```
        result = repeat(pattern, count)
```

```
        assert result == expected, f'repeat('{pattern}', {count}) = '{result}', ожидалось '{expected}'
```

```
    @pytest.mark.parametrize("pattern,count,expected_error,error_msg", [
```



```

("e", -2, ValueError, "repeat_count не может быть отрицательным"),
(None, 1, TypeError, "pattern должен быть строкой"),
("abc", -1, ValueError, "repeat_count не может быть отрицательным"),
(123, 1, TypeError, "pattern должен быть строкой"),
("abc", 2.5, TypeError, "repeat_count должен быть целым числом"),
([1, 2], 1, TypeError, "pattern должен быть строкой"),
("test", "3", TypeError, "repeat_count должен быть целым числом"),
(True, 5, TypeError, "pattern должен быть строкой"),
])

def test_repeat_invalid_cases(self, pattern, count, expected_error, error_msg):
    """Параметризованный тест некорректных случаев"""
    with pytest.raises(expected_error, match=error_msg):
        repeat(pattern, count)

```

```

@pytest.mark.parametrize("count", [0, 1, 2, 5, 10, 100])

```

```

def test_repeat_various_counts(self, count):
    """Тест с разным количеством повторений"""
    result = repeat("x", count)
    assert len(result) == count
    assert result == "x" * count

```

```

# =====
# Фикстуры
# =====

```

```

@pytest.fixture
def simple_pattern():
    """Простой паттерн для тестов"""
    return "abc"

```

```

@pytest.fixture

```

```

def complex_pattern():
    """Сложный паттерн для тестов"""
    return "Hello World! "

def test_repeat_with_simple_fixture(simple_pattern):
    """Тест с простой фикстурой"""
    result = repeat(simple_pattern, 3)
    assert result == "abcabcabc"
    assert len(result) == 9

def test_repeat_with_complex_fixture(complex_pattern):
    """Тест со сложной фикстурой"""
    result = repeat(complex_pattern, 2)
    assert result == "Hello World! Hello World! "

# =====
# Интеграционные тесты
# =====

def test_repeat_in_string_operations():
    """Использование repeat в строковых операциях"""
    pattern = "Ha"
    laughter = repeat(pattern, 3)
    assert laughter + "!" == "HaHaHa!"
    assert len(laughter) == 6

def test_repeat_with_concatenation():
    """Конкатенация с repeat"""
    part1 = repeat("AB", 2) # ABAB
    part2 = repeat("CD", 3) # CDCDCD
    assert part1 + part2 == "ABAB CDCDCD"

```

```

def test_repeat_to_build_pyramid():
    """Построение пирамиды с помощью repeat"""
    lines = []
    for i in range(1, 4):
        spaces = repeat(" ", 3 - i)
        stars = repeat("*", 2 * i - 1)
        lines.append(spaces + stars)
    expected = [
        "  *",
        " ***",
        "*****"
    ]
    assert lines == expected

# =====
# Демонстрация с объяснением ошибки в спецификации
# =====

def demonstrate_repeat():
    """Демонстрация работы функции repeat с объяснением"""
    print("Демонстрация работы repeat:\n")

    print(f"repeat('e', 0) = '{repeat('e', 0)}'")
    print(f"repeat('e', 3) = '{repeat('e', 3)}'")

    # Показываем правильный результат
    pattern = " ABC "
    result = repeat(pattern, 2)
    print(f"\nПаттерн: '{pattern}' (длина {len(pattern)})")
    print(f"Паттерн * 2: '{pattern * 2}'")

```

```
print(frepeat("{pattern}", 2) = "{result}")
print(f'Длина результата: {len(result)}')
print(f'\nОбъяснение:')
print(f" {pattern}" + "{pattern}" = "{pattern + pattern}")
print(f" В середине два пробела подряд: "{" ")")
print(f'\n Спецификация содержит ошибку:')
print(f" Спецификация говорит: " ABCABC ")
print(f" Правильный результат: " ABC ABC ")
```

```
try:
    repeat("e", -2)
except ValueError as e:
    print(f'\nrepeat("e", -2) -> ValueError: {e}')
```

```
try:
    repeat(None, 1)
except TypeError as e:
    print(f'repeat(None, 1) -> TypeError: {e}')
```

```
print(f'\nrepeat("Hello", 3) = "{repeat("Hello", 3)}"')
print(f'repeat("😊", 5) = "{repeat("😊", 5)}"')
print(f'repeat("AB", 4) = "{repeat("AB", 4)}"')
```

```
# =====
# Запуск тестов
# =====
```

```
if __name__ == "__main__":
    import sys
```

```
if "--demo" in sys.argv:
```

demonstrate\_repeat()

else:

pytest.main([\_\_file\_\_, "-v", "--tb=short"])

```
C:\Windows\system32\cmd.exe
test_repeat.py::TestRepeatParametrized::test_repeat_invalid_cases[None-1-TypeError-pattern \u0434\u043e\u043b\u0436\u0435\u043d\u0434\u0431\u0442\u044c \u0441\u0440\u0435\u0434\u043e\u0439] PASSED [ 77%]
test_repeat.py::TestRepeatParametrized::test_repeat_invalid_cases[abc--1-ValueError-repeat_count \u0434\u0435 \u043c\u043e\u0436\u0435\u043d\u0434\u0431\u0442\u044c \u0441\u0440\u0435\u0434\u043e\u0439] PASSED [ 78%]
test_repeat.py::TestRepeatParametrized::test_repeat_invalid_cases[123-1-TypeError-pattern \u0434\u043e\u043b\u0436\u0435\u043d\u0434\u0431\u0442\u044c \u0441\u0440\u0435\u0434\u043e\u0439] PASSED [ 79%]
test_repeat.py::TestRepeatParametrized::test_repeat_invalid_cases[abc-2.5-TypeError-repeat_count \u0434\u0435 \u043c\u043e\u0436\u0435\u043d\u0434\u0431\u0442\u044c \u0441\u0440\u0435\u0434\u043e\u0439] PASSED [ 81%]
test_repeat.py::TestRepeatParametrized::test_repeat_invalid_cases[pattern5-1-TypeError-pattern \u0434\u043e\u043b\u0436\u0435\u043d\u0434\u0431\u0442\u044c \u0441\u0440\u0435\u0434\u043e\u0439] PASSED [ 82%]
test_repeat.py::TestRepeatParametrized::test_repeat_invalid_cases[test-3-TypeError-repeat_count \u0434\u0435 \u043c\u043e\u0436\u0435\u043d\u0434\u0431\u0442\u044c \u0441\u0440\u0435\u0434\u043e\u0439] PASSED [ 83%]
test_repeat.py::TestRepeatParametrized::test_repeat_invalid_cases[True-5-TypeError-pattern \u0434\u043e\u043b\u0436\u0435\u043d\u0434\u0431\u0442\u044c \u0441\u0440\u0435\u0434\u043e\u0439] PASSED [ 85%]
test_repeat.py::TestRepeatParametrized::test_repeat_various_counts[0] PASSED [ 86%]
test_repeat.py::TestRepeatParametrized::test_repeat_various_counts[1] PASSED [ 87%]
test_repeat.py::TestRepeatParametrized::test_repeat_various_counts[2] PASSED [ 89%]
test_repeat.py::TestRepeatParametrized::test_repeat_various_counts[5] PASSED [ 90%]
test_repeat.py::TestRepeatParametrized::test_repeat_various_counts[10] PASSED [ 91%]
test_repeat.py::TestRepeatParametrized::test_repeat_various_counts[100] PASSED [ 93%]
test_repeat.py::test_repeat_with_simple_fixture PASSED [ 94%]
test_repeat.py::test_repeat_with_complex_fixture PASSED [ 95%]
test_repeat.py::test_repeat_in_string_operations PASSED [ 97%]
test_repeat.py::test_repeat_with_concatenation PASSED [ 98%]
test_repeat.py::test_repeat_to_build_pyramid PASSED [100%]

===== 74 passed in 0.14s =====
Для продолжения нажмите любую клавишу . . .
```

Вывод: были отработаны базовые знания языка программирования Python при решении практических задач